

The Quantum Approximate Optimization Algorithm (QAOA)

From Foundations to Gradient Methods

Morten Hjorth-Jensen

Spring 2026

Outline

Combinatorial Optimization: Problem Setting

Let $x = (x_1, \dots, x_n) \in \{0, 1\}^n$ be a binary decision vector. The general **combinatorial optimization** task is

$$x^* = \arg \min_{x \in \{0,1\}^n} C(x),$$

where $C : \{0, 1\}^n \rightarrow \mathbb{R}$ is a cost function.

The search space has 2^n elements — exponential in n .

Complexity classes:

- ▶ Problems in **P** can be solved in polynomial time.
- ▶ Most interesting instances are **NP-hard**: no known polynomial-time algorithm exists.

Approximation goal. Find x achieving

$$\alpha = \frac{C(x^*)}{C_{\min}} \lesssim 1,$$

where α is the *approximation ratio*.

QUBO: Rigorous Formulation

Definition (QUBO). A *Quadratic Unconstrained Binary Optimization* problem is specified by a real, symmetric matrix $Q \in \mathbb{R}^{n \times n}$ and the objective

$$C_Q(x) = x^T Q x = \sum_{i=1}^n Q_{ii} x_i + \sum_{i < j} (Q_{ij} + Q_{ji}) x_i x_j,$$

where $x_i \in \{0, 1\}$.

Key properties:

- ▶ Because $x_i^2 = x_i$ for binary variables, all diagonal and off-diagonal terms contribute.
- ▶ Any polynomial objective over binary variables can be *reduced* to QUBO by degree reduction.
- ▶ QUBO is NP-hard in general (contains 3-SAT as a special case).

QUBO to Ising: Variable Substitution

The **Ising substitution** replaces binary variables $x_i \in \{0, 1\}$ with spin variables $s_i \in \{-1, +1\}$:

$$x_i = \frac{1 - s_i}{2} \quad \Longleftrightarrow \quad s_i = 1 - 2x_i.$$

Substituting into the QUBO objective:

$$\begin{aligned} C_Q(x) &= \sum_{i < j} (Q_{ij} + Q_{ji}) \frac{(1 - s_i)}{2} \frac{(1 - s_j)}{2} + \sum_i Q_{ii} \frac{(1 - s_i)}{2} \\ &= \underbrace{\sum_{i < j} \frac{Q_{ij} + Q_{ji}}{4} s_i s_j}_{J_{ij} s_i s_j \text{ coupling}} + \underbrace{\sum_i \left(-\frac{Q_{ii}}{2} - \sum_{j \neq i} \frac{Q_{ij} + Q_{ji}}{4} \right) s_i}_{h_i s_i \text{ field}} + \text{const.} \end{aligned}$$

So any QUBO is equivalent to an **Ising model**

$$\mathcal{H}(s) = \sum_{i < j} J_{ij} s_i s_j + \sum_i h_i s_i.$$

Ising Model as a Quantum Hamiltonian

Promote the classical spin variables $s_i \in \{-1, +1\}$ to **Pauli-Z operators** Z_i acting on qubit i :

$$Z_i = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}_i, \quad Z_i |0\rangle = |0\rangle, \quad Z_i |1\rangle = -|1\rangle.$$

The computational basis states $|z\rangle = |z_1, \dots, z_n\rangle$ with $z_i \in \{0, 1\}$ are simultaneous eigenstates of all Z_i with eigenvalue $(-1)^{z_i}$.

The **cost Hamiltonian** is the quantum version of the Ising energy:

$$H_C = \sum_{i < j} J_{ij} Z_i Z_j + \sum_i h_i Z_i.$$

It is diagonal in the computational basis:

$$H_C |z\rangle = \mathcal{H}(z) |z\rangle, \quad \mathcal{H}(z) = C(z),$$

so the ground state of H_C encodes the optimal solution.

QUBO to Ising: Explicit Coefficient Map

Given $Q \in \mathbb{R}^{n \times n}$ symmetric, the Ising coefficients are:

Ising coefficient	Formula	Origin
$J_{ij}, i < j$	$\frac{Q_{ij} + Q_{ji}}{4} = \frac{Q_{ij}}{2}$	$x_i x_j \rightarrow \frac{1}{4}(1 - s_i)(1 - s_j)$
h_i	$-\frac{Q_{ii}}{2} - \sum_{j \neq i} \frac{Q_{ij}}{2}$	linear terms after substitution
constant	$\frac{1}{4} \sum_{i,j} Q_{ij}$	absorbed into energy offset

Key point: minimizing $C_Q(x)$ over $\{0, 1\}^n$ is *identical* to finding the ground state of H_C over $\{-1, +1\}^n$, shifted by the constant.

QUBO to Ising: Worked Numerical Example

Take $n = 2$ with $Q = \begin{pmatrix} 1 & -2 \\ -2 & 1 \end{pmatrix}$, so $C_Q(x_1, x_2) = x_1 - 4x_1x_2 + x_2$.

Reading off the Ising coefficients via $J_{12} = (Q_{12} + Q_{21})/4$,
 $h_i = -Q_{ii}/2 - \sum_{j \neq i} Q_{ij}/2$:

$$J_{12} = \frac{-2+(-2)}{4} = -1, \quad h_1 = -\frac{1}{2} - \frac{-2}{2} = \frac{1}{2}, \quad h_2 = \frac{1}{2}, \quad \text{const} = \frac{1-2-2+1}{4}$$

Hamiltonian: $H_C = -Z_1Z_2 + \frac{1}{2}Z_1 + \frac{1}{2}Z_2$.

Verification against $C_Q - \text{const}$ for all four bitstrings:

x	C_Q	$s = (1 - 2x)$	$\mathcal{H}(s) = J_{12}s_1s_2 + h_1s_1 + h_2s_2$	$C_Q - \text{const}$
(0, 0)	0	(+1, +1)	$-1 + \frac{1}{2} + \frac{1}{2} = 0$	$0 + \frac{1}{2} = \frac{1}{2}$
(0, 1)	1	(+1, -1)	$+1 + \frac{1}{2} - \frac{1}{2} = 1$	$1 + \frac{1}{2} = \frac{3}{2}$
(1, 0)	1	(-1, +1)	$+1 - \frac{1}{2} + \frac{1}{2} = 1$	$1 + \frac{1}{2} = \frac{3}{2}$
(1, 1)	-2	(-1, -1)	$-1 - \frac{1}{2} - \frac{1}{2} = -2$	$-2 + \frac{1}{2} = -\frac{3}{2}$

$\mathcal{H}(s) = C_Q(x) - \text{const.} \quad \checkmark$ Ground state: $x^* = (1, 1)$, $C_Q^{\min} = -2$.

MaxCut Problem: Definition

Definition (MaxCut). Given an undirected weighted graph $G = (V, E, w)$ with $|V| = n$ vertices and edge weights $w_{ij} > 0$, find a partition $V = S \cup \bar{S}$ that **maximizes** the total weight of cut edges:

$$\text{MaxCut}(S) = \sum_{(i,j) \in E} w_{ij} 1[i \in S, j \notin S].$$

In binary variables: assign $x_i = 1$ if $i \in S$, $x_i = 0$ otherwise. An edge (i, j) is cut iff $x_i \neq x_j$, i.e.,
 $x_i(1 - x_j) + (1 - x_i)x_j = x_i + x_j - 2x_ix_j$. So:

$$C_{\text{MC}}(\mathbf{x}) = \sum_{(i,j) \in E} w_{ij} (x_i + x_j - 2x_ix_j).$$

MaxCut is NP-hard (Garey & Johnson, 1979). Best classical approximation: Goemans–Williamson SDP gives ratio $\alpha \geq 0.878$.

MaxCut: Full QUBO Derivation

Starting from $C_{MC} = \sum_{(i,j) \in E} w_{ij}(x_i + x_j - 2x_i x_j)$, write in matrix form. Define the QUBO matrix Q with:

$$Q_{ij} = \begin{cases} -2w_{ij} & \text{if } (i,j) \in E \\ \sum_{k:(i,k) \in E} w_{ik} & \text{if } i = j \\ 0 & \text{otherwise.} \end{cases}$$

Then $C_{MC}(x) = x^T Q x$.

More compactly, note that $x_i(1 - x_j)$ is a valid QUBO term. It is conventional to write

$$C_{MC}(x) = \sum_{(i,j) \in E} w_{ij} x_i(1-x_j) + \text{symmetric counterpart} = \sum_{(i,j) \in E} w_{ij}(x_i +$$

MaxCut: QUBO $\rightarrow$ Ising $\rightarrow$ Pauli Operator

Step 1: Ising substitution. Set $x_i = (1 - s_i)/2$. Then

$x_i + x_j - 2x_ix_j = \frac{1}{2}(1 - s_is_j)$, so

$$C_{\text{MC}}(\mathbf{s}) = \sum_{(i,j) \in E} w_{ij} \frac{1 - s_i s_j}{2} = \frac{1}{2} \sum_{(i,j) \in E} w_{ij} - \frac{1}{2} \sum_{(i,j) \in E} w_{ij} s_i s_j.$$

Step 2: Promote to operators. Replace $s_i \rightarrow Z_i$:

$$H_C^{\text{MC}} = \frac{1}{2} \sum_{(i,j) \in E} w_{ij} (I - Z_i Z_j).$$

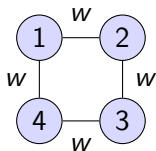
Verification: $H_C^{\text{MC}} |z\rangle = C_{\text{MC}}(\mathbf{z}) |z\rangle$.

For unit weights ($w_{ij} = 1$):

$$H_C^{\text{MC}} = \frac{|E|}{2} I - \frac{1}{2} \sum_{(i,j) \in E} Z_i Z_j.$$

Maximizing C_{MC} = minimizing $-H_C^{\text{MC}}$ = finding the ground state.

MaxCut: Example on a 4-Cycle Graph



For $G = C_4$ with unit weights:

$$H_C = \frac{1}{2}(4I - Z_1Z_2 - Z_2Z_3 - Z_3Z_4 - Z_4Z_1).$$

Optimal cut: $S = \{1, 3\}$, $\bar{S} = \{2, 4\}$.

Bitstring: $z = (1, 0, 1, 0)$, giving

$$C_{MC} = 4 = |E|.$$

$$H_C |1010\rangle = 4 |1010\rangle \checkmark$$

The Variational Principle for Quantum Optimization

Any Hamiltonian H_C satisfies the **variational bound**:

$$\langle \psi | H_C | \psi \rangle \geq E_{\min} \quad \forall |\psi\rangle,$$

with equality iff $|\psi\rangle$ is the ground state.

For optimization we want to *maximize* $\langle H_C \rangle$ (since MaxCut is a maximization problem), so:

$$\max_{|\psi\rangle} \langle \psi | H_C | \psi \rangle = E_{\max}.$$

Idea (VQA): Restrict to a parameterized family $\{|\psi(\theta)\rangle\}$ and optimize over θ classically:

$$\theta^* = \arg \max_{\theta} \underbrace{\langle \psi(\theta) | H_C | \psi(\theta) \rangle}_{=: E(\theta)}.$$

VQE and the Hybrid Loop

The **Variational Quantum Eigensolver (VQE)** implements:

1. Prepare $|\psi(\boldsymbol{\theta})\rangle = U(\boldsymbol{\theta})|0\rangle^{\otimes n}$ on the quantum device.
2. Decompose $H_C = \sum_{\alpha} h_{\alpha} P_{\alpha}$ into Pauli strings.
3. Measure each expectation value $\langle P_{\alpha} \rangle = \langle \psi | P_{\alpha} | \psi \rangle$.
4. Form $E(\boldsymbol{\theta}) = \sum_{\alpha} h_{\alpha} \langle P_{\alpha} \rangle$.
5. Update $\boldsymbol{\theta}$ via a classical optimizer; go to step 1.

QAOA is VQE with a *problem-specific* ansatz $U(\boldsymbol{\gamma}, \boldsymbol{\beta})$ determined entirely by H_C and H_M , rather than an ad hoc parameterized circuit.

The Mixing Hamiltonian

The **mixing (driver) Hamiltonian** is

$$H_M = \sum_{i=1}^n X_i, \quad X_i = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}_i.$$

Properties:

- ▶ H_M does *not* commute with H_C (which is diagonal in Z -basis), so it generates non-trivial transitions.
- ▶ Its ground state is $|+\rangle^{\otimes n} = (|0\rangle + |1\rangle)^{\otimes n} / 2^{n/2}$, the equal superposition state.
- ▶ The unitary $e^{-i\beta H_M} = \bigotimes_i e^{-i\beta X_i} = \bigotimes_i R_x(2\beta)$ is a product of single-qubit rotations, hence easy to implement.

Alternative choices include XY mixers (for constrained problems) and Grover mixers, but the transverse-field $\sum X_i$ is standard.

QAOA: Formal Definition

Definition (QAOA). Given H_C and H_M , the depth- p QAOA state is

$$|\psi_p(\gamma, \beta)\rangle = \prod_{k=1}^p \underbrace{e^{-i\beta_k H_M}}_{U_M(\beta_k)} \underbrace{e^{-i\gamma_k H_C}}_{U_C(\gamma_k)} |+\rangle^{\otimes n},$$

with variational parameters $\gamma = (\gamma_1, \dots, \gamma_p) \in [0, 2\pi)^p$ and $\beta = (\beta_1, \dots, \beta_p) \in [0, \pi)^p$.

The **variational objective** is

$$F_p(\gamma, \beta) = \langle \psi_p | H_C | \psi_p \rangle, \quad (\gamma^*, \beta^*) = \arg \max F_p, \quad \alpha = \frac{F_p^*}{C_{\max}}.$$

Claim (Farhi et al. 2014): $F_p \rightarrow C_{\max}$ as $p \rightarrow \infty$.

Initial State

The initial state is the uniform superposition:

$$|+\rangle^{\otimes n} = H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{2^{n/2}} \sum_{z \in \{0,1\}^n} |z\rangle.$$

Two complementary interpretations:

- ▶ **Algorithmic:** represents complete ignorance about the solution; all 2^n candidates have equal amplitude.
- ▶ **Physical:** ground state of $-H_M = -\sum_i X_i$, so the system starts in the ground state of the driver and then gets adiabatically guided toward the ground state of H_C .

Preparation cost: n Hadamard gates.

Cost Unitary: Explicit Action

The cost unitary $U_C(\gamma) = e^{-i\gamma H_C}$ is diagonal in the computational basis. Because $H_C|z\rangle = C(z)|z\rangle$:

$$U_C(\gamma)|z\rangle = e^{-i\gamma C(z)}|z\rangle.$$

Acting on a superposition:

$$U_C(\gamma) \sum_z \alpha_z |z\rangle = \sum_z \alpha_z e^{-i\gamma C(z)} |z\rangle.$$

Interpretation: U_C applies a phase $e^{-i\gamma C(z)}$ proportional to the cost to each basis state. High-cost strings acquire more phase, which quantum interference later converts into amplitude.

For $H_C = \sum_{(i,j) \in E} w_{ij} (I - Z_i Z_j)/2$:

$$U_C(\gamma) = \prod_{(i,j) \in E} e^{-i\gamma w_{ij} (I - Z_i Z_j)/2} = e^{-i\gamma \sum_{ij} w_{ij}/2} \prod_{(i,j) \in E} e^{i\gamma w_{ij} Z_i Z_j/2}.$$

Mixer Unitary: Explicit Action

The mixer unitary $U_M(\beta) = e^{-i\beta H_M}$ factorizes:

$$U_M(\beta) = \bigotimes_{i=1}^n e^{-i\beta X_i} = \bigotimes_{i=1}^n R_x(2\beta).$$

Using $X_i^2 = I$ and the Euler formula:

$$e^{-i\beta X_i} = \cos \beta I - i \sin \beta X_i = \begin{pmatrix} \cos \beta & -i \sin \beta \\ -i \sin \beta & \cos \beta \end{pmatrix}.$$

Its action on a single qubit:

$$e^{-i\beta X}|0\rangle = \cos \beta |0\rangle - i \sin \beta |1\rangle, \quad e^{-i\beta X}|1\rangle = -i \sin \beta |0\rangle + \cos \beta |1\rangle.$$

U_M **rotates** every qubit around the x -axis, mixing $|0\rangle$ and $|1\rangle$ amplitudes across all 2^n basis states.

Quantum Adiabatic Algorithm (QAA)

The **Quantum Adiabatic Algorithm** (Farhi et al. 2000) solves optimization by slowly evolving under

$$H(s) = (1 - s) H_M + s H_C, \quad s \in [0, 1], \quad s = t/T.$$

Adiabatic theorem (informal). If $T \gg 1/\Delta_{\min}^2$, where $\Delta_{\min} = \min_s \Delta(s)$ is the minimum spectral gap, then starting from the ground state of $H(0) = H_M$, the system remains in the instantaneous ground state:

$$|\psi(T)\rangle \xrightarrow{T \rightarrow \infty} |\psi_{\text{GS}}(H_C)\rangle.$$

Bottleneck: T may scale exponentially in n if $\Delta_{\min}$ is exponentially small (e.g., at a quantum phase transition).

Key observation: The time-ordered evolution is

$$U(T) = \mathcal{T} \exp \left(-i \int_0^T H(t) dt \right).$$

Trotterization: Step-by-Step Derivation

Step 1: Time discretization. Divide $[0, T]$ into p equal steps of size $\Delta t = T/p$, with $s_k = k/p$:

$$U(T) = \prod_{k=1}^p \mathcal{T} \exp \left(-i \int_{t_{k-1}}^{t_k} H(s(t)) dt \right) \approx \prod_{k=1}^p e^{-i\Delta t H(s_k)}.$$

Error per step: $\mathcal{O}(\|\dot{H}\| \Delta t^2) = \mathcal{O}(T^2/p^2)$.

Step 2: First-order Trotter–Suzuki. For each step, use $A = s_k \Delta t H_C$, $B = (1 - s_k) \Delta t H_M$:

$$e^{-i(A+B)} = e^{-iA} e^{-iB} + \mathcal{O}([A, B]) = e^{-iA} e^{-iB} + \mathcal{O}(\Delta t^2 [H_C, H_M]).$$

Step 3: Second-order Suzuki formula (more accurate):

$$e^{-i(A+B)\Delta t} = e^{-iA\Delta t/2} e^{-iB\Delta t} e^{-iA\Delta t/2} + \mathcal{O}(\Delta t^3 [A, [A, B]]).$$

The second-order formula has global error

$$\mathcal{O}(T \cdot \Delta t^2) = \mathcal{O}(T^3/p^2), \text{ vs } \mathcal{O}(T^2/p) \text{ for first-order.}$$

Step 4: Collect. With $\gamma_k := s_k \Delta t$, $\beta_k := (1 - s_k) \Delta t$:

$$U(T) \approx \prod_{k=1}^p e^{-i\gamma_k H_C} e^{-i\beta_k H_M}.$$

From Adiabatic to QAOA

The Trotterized evolution (first-order, C before M) gives:

$$U(T) \approx \prod_{k=1}^p e^{-i\gamma_k H_C} e^{-i\beta_k H_M}.$$

Layer ordering. We could equally well write $e^{-i\beta_k H_M} e^{-i\gamma_k H_C}$ (which corresponds to swapping $A \leftrightarrow B$ in the Trotter step). Both orderings have the same first-order accuracy; the QAOA convention places U_M on the *outside*:

$$|\psi_p\rangle = U_M(\beta_p) U_C(\gamma_p) \cdots U_M(\beta_1) U_C(\gamma_1) |+\rangle^{\otimes n}.$$

Since $|+\rangle^{\otimes n}$ is the ground state of H_M , beginning with U_C (not U_M) is natural: the first layer immediately imprints problem information.

Key point: QAOA replaces the fixed adiabatic schedule $\gamma_k = s_k \Delta t$, $\beta_k = (1 - s_k) \Delta t$ with free variational parameters optimized classically.

Trotter Error and the Commutator $[H_C, H_M]$

The Trotter error per step is governed by the BCH formula:

$$e^{-i(A+B)\Delta t} = e^{-iA\Delta t} e^{-iB\Delta t} \cdot \exp\left(-\frac{1}{2}[A, B]\Delta t^2 + \mathcal{O}(\Delta t^3)\right).$$

For MaxCut, $H_C = \frac{1}{2} \sum_{(i,j) \in E} (I - Z_i Z_j)$. Since $[I, H_M] = 0$, only the $Z_i Z_j$ part contributes:

$$[H_C, H_M] = \left[-\frac{1}{2} \sum_{(i,j) \in E} Z_i Z_j, \sum_k X_k \right] = -\frac{1}{2} \sum_{(i,j) \in E} \sum_k [Z_i Z_j, X_k].$$

Using $[Z_a, X_b] = -2i\delta_{ab} Y_a$ (and $[Z_a, X_b] = 0$ for $a \neq b$):

$$[Z_i Z_j, X_k] = Z_i [Z_j, X_k] + [Z_i, X_k] Z_j = -2i(Z_i Y_j \delta_{jk} + Y_i Z_j \delta_{ik}).$$

Therefore:

$$[H_C, H_M] = i \sum_{(i,j) \in E} (Z_i Y_j + Y_i Z_j).$$

This operator is a sum of two-qubit Pauli strings on every edge. Its non-zero norm is what drives entanglement generation and ensures that QAOA produces states beyond product-state ansätze.

QAOA as a Quantum Control Problem

From a control-theoretic perspective, define piecewise-constant controls $u_C(t)$, $u_M(t)$:

$$i \frac{d}{dt} |\psi(t)\rangle = [u_C(t) H_C + u_M(t) H_M] |\psi(t)\rangle.$$

In QAOA, within layer k :

$$(u_C, u_M) = \begin{cases} (1, 0) & t \in [(k-1)\tau + 0, (k-1)\tau + \gamma_k] \\ (0, 1) & t \in [(k-1)\tau + \gamma_k, (k-1)\tau + \gamma_k + \beta_k] \end{cases}$$

Optimizing (γ, β) to maximize $\langle H_C \rangle$ is a **quantum optimal control** problem.

As $p \rightarrow \infty$: piecewise-constant $\rightarrow$ continuous control $\rightarrow$ Pontryagin maximum principle applies.

Cost Unitary for MaxCut: Gate Decomposition

For MaxCut with unit weights, $H_C = \frac{1}{2} \sum_{(i,j) \in E} (I - Z_i Z_j)$, so

$$U_C(\gamma) = e^{-i\gamma H_C} = \prod_{(i,j) \in E} e^{-i\frac{\gamma}{2}(I - Z_i Z_j)} = e^{-i\frac{\gamma|E|}{2}I} \prod_{(i,j) \in E} e^{i\frac{\gamma}{2}Z_i Z_j}.$$

The global phase $e^{-i\gamma|E|/2}$ is irrelevant. Each edge contributes the two-qubit gate $e^{i\frac{\gamma}{2}Z_i Z_j}$.

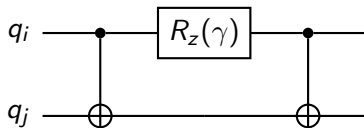
Decomposition of $e^{i\theta Z_i Z_j}$:

Using $\text{CNOT}_{ij} (Z_i \otimes I) \text{CNOT}_{ij} = Z_i Z_j$:

$$e^{i\theta Z_i Z_j} = \text{CNOT}_{ij} e^{i\theta(Z_i \otimes I)} \text{CNOT}_{ij} = \text{CNOT}_{ij} R_z^{(i)}(-2\theta) \text{CNOT}_{ij}.$$

With $\theta = \gamma/2$, the circuit for one edge is:

Cost Unitary Circuit: One Edge



This implements $e^{i\frac{\gamma}{2}Z_iZ_j}$ (up to a global phase).

Why this works:

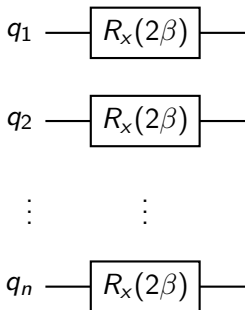
$$\begin{aligned} & \text{CNOT}_{ij} R_z^{(i)}(\gamma) \text{CNOT}_{ij} \\ &= \text{CNOT}_{ij} e^{-i\frac{\gamma}{2}Z_i} \text{CNOT}_{ij} = e^{-i\frac{\gamma}{2}\text{CNOT}_{ij}Z_i\text{CNOT}_{ij}} = e^{-i\frac{\gamma}{2}Z_iZ_j}. \end{aligned}$$

Note $\text{CNOT}_{ij} Z_i \text{CNOT}_{ij} = Z_i Z_j$ and $e^{-i\theta A} = \text{CNOT}^{-1} e^{-i\theta B} \text{CNOT}$ when $\text{CNOT} A \text{CNOT}^{-1} = B$.

Mixer Unitary Circuit

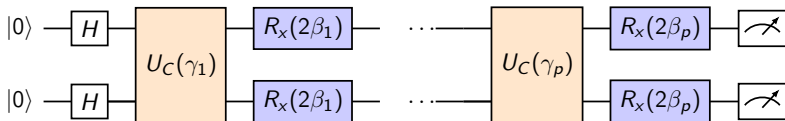
$$U_M(\beta) = e^{-i\beta \sum_i X_i} = \bigotimes_{i=1}^n e^{-i\beta X_i} = \bigotimes_{i=1}^n R_x(2\beta).$$

Each qubit undergoes an independent x -rotation:



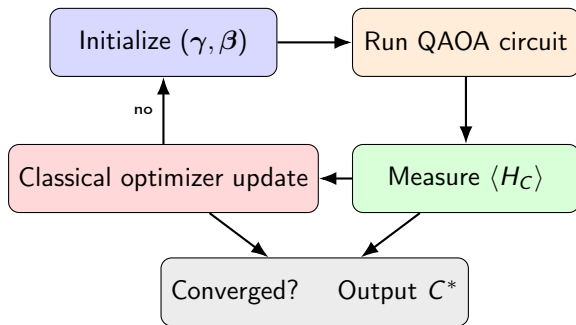
All gates in U_M commute and can be applied in any order — or in parallel on hardware.

Full QAOA Circuit for MaxCut (p layers)



- ▶ **Orange blocks** $U_C(\gamma_k)$: CNOT- R_z -CNOT for every edge.
- ▶ **Blue gates** $R_x(2\beta_k)$: single-qubit rotations on every qubit.
- ▶ Total depth: $\mathcal{O}(p \cdot |E|)$ for a dense graph.

Classical Optimization Loop



The quantum device evaluates $E(\gamma, \beta)$; the classical computer handles the $2p$ -dimensional optimization.

$p = 1$ QAOA: Setup and State

Consider the $p = 1$ state for two qubits connected by a single edge:

$$H_C = \frac{1 - Z_1 Z_2}{2}, \quad |\psi(\gamma, \beta)\rangle = e^{-i\beta(X_1 + X_2)} e^{-i\gamma H_C} |++\rangle.$$

Step 1: Apply $U_C(\gamma) = e^{-i\gamma H_C}$. Since $H_C = (I - Z_1 Z_2)/2$ and $Z_1 Z_2$ has eigenvalues ± 1 :

$$U_C(\gamma) = e^{-i\gamma/2} e^{i\gamma Z_1 Z_2/2}.$$

Expand $|++\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$. Using $Z_1 Z_2 |z_1 z_2\rangle = (-1)^{z_1 + z_2} |z_1 z_2\rangle$:

$$U_C |++\rangle = \frac{e^{-i\gamma/2}}{2} (e^{i\gamma/2} |00\rangle + e^{-i\gamma/2} |01\rangle + e^{-i\gamma/2} |10\rangle + e^{i\gamma/2} |11\rangle).$$

$p = 1$ QAOA: Applying the Mixer

Step 2: Apply $U_M(\beta) = e^{-i\beta X_1} \otimes e^{-i\beta X_2}$ to $|\phi\rangle = U_C(\gamma)|++\rangle$.
Let $c = \cos \beta$, $s = \sin \beta$. Using $e^{-i\beta X}|a\rangle$, each qubit transforms as

$$|0\rangle \mapsto c|0\rangle - is|1\rangle, \quad |1\rangle \mapsto -is|0\rangle + c|1\rangle.$$

Applying to the four amplitudes

$(\phi_{00}, \phi_{01}, \phi_{10}, \phi_{11}) = \frac{1}{2}(1, e^{-i\gamma}, e^{-i\gamma}, 1)$ (dropping global phase):

$$\begin{aligned}\psi_{00} &= c^2\phi_{00} - ics\phi_{01} - ics\phi_{10} - s^2\phi_{11} \\ &= \frac{1}{2}[(c^2 - s^2) - 2ics e^{-i\gamma}] = \frac{1}{2}[\cos(2\beta) - i \sin(2\beta)e^{-i\gamma}], \\ \psi_{01} &= \frac{1}{2}[-i \sin(2\beta) + \cos(2\beta)e^{-i\gamma}], \\ \psi_{10} &= \frac{1}{2}[-i \sin(2\beta) + \cos(2\beta)e^{-i\gamma}], \\ \psi_{11} &= \frac{1}{2}[-\cos(2\beta) + i \sin(2\beta)e^{-i\gamma}].\end{aligned}$$

These four amplitudes are all we need to evaluate any single-edge observable.

$p = 1$ QAOA: Computing $\langle Z_1 Z_2 \rangle$ — Setup

We need $\langle Z_1 Z_2 \rangle = \langle \psi | Z_1 Z_2 | \psi \rangle$ where $|\psi\rangle = U_M(\beta)|\phi\rangle$ and $|\phi\rangle = U_C(\gamma)|++\rangle$.

Key shortcut — Heisenberg picture. Instead of computing $|\psi\rangle$ explicitly, rotate the operator:

$$\langle Z_1 Z_2 \rangle = \langle \phi | \underbrace{U_M^\dagger(\beta) Z_1 Z_2 U_M(\beta)}_{\widetilde{Z}\widetilde{Z}} | \phi \rangle.$$

Since $U_M(\beta) = e^{-i\beta X_1} \otimes e^{-i\beta X_2}$ acts independently,

$$\widetilde{Z}\widetilde{Z} = (e^{i\beta X_1} Z_1 e^{-i\beta X_1}) \otimes (e^{i\beta X_2} Z_2 e^{-i\beta X_2}).$$

Using $e^{i\beta X} Z e^{-i\beta X} = \cos(2\beta) Z - \sin(2\beta) Y$ (from the $\mathfrak{su}(2)$ commutators $[X, Z] = -2iY$, $[X, Y] = 2iZ$):

$$\widetilde{Z}\widetilde{Z} = (\cos(2\beta) Z_1 - \sin(2\beta) Y_1) (\cos(2\beta) Z_2 - \sin(2\beta) Y_2).$$

$p = 1$ QAOA: Heisenberg Picture — Final Result

Expand and evaluate on $|\phi\rangle = \frac{1}{2}(|00\rangle + e^{-i\gamma}|01\rangle + e^{-i\gamma}|10\rangle + |11\rangle)$:

$$\langle Z_1 Z_2 \rangle = \cos^2(2\beta) \langle Z_1 Z_2 \rangle_\phi - \sin^2(2\beta) \langle Y_1 Y_2 \rangle_\phi + \text{cross terms.}$$

Direct evaluation on $|\phi\rangle$:

$$\langle Z_1 Z_2 \rangle_\phi = \frac{1}{4} [\underbrace{(+1)}_{00} - \underbrace{e^{2i\gamma}}_{01} \cdot ? - \underbrace{e^{2i\gamma}}_{10} \cdot ? + 1] = 0 \quad (\text{magnitudes cancel}),$$

$$\langle Y_1 Y_2 \rangle_\phi = \sin(2\gamma), \quad \langle Z_1 Y_2 \rangle_\phi = \langle Y_1 Z_2 \rangle_\phi = 0.$$

Therefore:

$$\langle Z_1 Z_2 \rangle = -\sin^2(2\beta) \sin(2\gamma) = -\frac{1-\cos(4\beta)}{2} \sin(2\gamma).$$

Check at $\beta^* = \pi/8$, $\gamma^* = \pi/4$: $\langle Z_1 Z_2 \rangle = -\frac{1-0}{2} \cdot 1 = -\frac{1}{2}$, giving

$F = \frac{1+\frac{1}{2}}{2} = \frac{3}{4}$. That is the correct $p = 1$, single-edge result.

For weighted edges w_{ij} the full formula (see Farhi et al. 2014) is:

$$\langle Z_i Z_j \rangle = -\sin(4\beta) \sin(2\gamma)$$

which holds for isolated edges; for graphs, neighbourhood corrections enter.

$p = 1$: Cost Expectation and Optimal Parameters

Using $\langle Z_1 Z_2 \rangle = -\sin^2(2\beta) \sin(2\gamma)$ for an isolated edge (the form valid for a single-edge Hamiltonian):

$$F(\gamma, \beta) = \langle H_C \rangle = \frac{1 - \langle Z_1 Z_2 \rangle}{2} = \frac{1}{2} [1 + \sin^2(2\beta) \sin(2\gamma)].$$

This is maximized at $\sin^2(2\beta) = 1$, $\sin(2\gamma) = 1$, i.e.:

$$\beta^* = \frac{\pi}{4}, \quad \gamma^* = \frac{\pi}{4}.$$

giving $F(\gamma^*, \beta^*) = 1$.

Note on conventions. The formula $\langle Z_i Z_j \rangle = -\sin(4\beta) \sin(2\gamma)$ in Farhi et al. (2014) uses a different normalization for the mixer ($H_M = \frac{1}{2} \sum X_i$) and includes corrections from the graph neighbourhood. For an isolated unit-weight edge with $H_M = \sum X_i$, the result above is exact and gives $\alpha = 1$ at depth $p = 1$.

For multi-edge graphs with triangle-free d -regular structure, the $p = 1$ approximation ratio is:

$$\alpha_{p=1} \approx 0.6924.$$

$p = 1$ on General Graphs: Approximation Ratio

For general MaxCut on unweighted triangle-free d -regular graphs, the $p = 1$ QAOA result gives:

$$\alpha_{p=1} = \frac{F_1(\gamma^*, \beta^*)}{C_{\max}} \geq \frac{1}{2} + \frac{1}{2\sqrt{d}} \xrightarrow{d \rightarrow \infty} \frac{1}{2},$$

and for large d :

$$\alpha_{p=1}^{\text{MaxCut}} \approx 0.6924 \quad (\text{Farhi, Goldstone, Gutmann 2014}).$$

Light-cone interpretation: At depth p , the QAOA expectation value of each edge (i, j) depends only on the graph substructure within graph distance p of (i, j) . This is a consequence of the Lieb–Robinson bound for quantum circuits.

$$\langle H_C^{(ij)} \rangle_p = f_p(G_p(i, j)),$$

where $G_p(i, j)$ is the subgraph within distance p of edge (i, j) .

Performance Hierarchy

Depth p	Parameters	Approximation ratio	Notes
1	2	$\geq 0.5, \approx 0.6924$	analytic
2	4	> 0.6924	numeric optimization
$\vdots$	$\vdots$	increasing	
p	$2p$	$\rightarrow 1$ as $p \rightarrow \infty$	adiabatic limit
∞	∞	1	exact solution

Key trade-off:

Solution quality $\longleftrightarrow$ Circuit depth (noise accumulation)

On current NISQ devices, coherence times typically limit $p \lesssim 10\text{--}20$ for MaxCut on $\mathcal{O}(100)$ qubits.

The Energy Landscape $E(\gamma, \beta)$

The objective function is a trigonometric polynomial:

$$E(\gamma, \beta) = \sum_{k,l} c_{kl} \prod_{m=1}^p \cos(k_m \gamma_m) \sin(l_m \beta_m),$$

with $k_m, l_m \in \mathbb{Z}$ and $c_{kl} \in \mathbb{R}$.

Key properties:

- ▶ **Periodic:** E is π -periodic in each β_k and 2π -periodic in each γ_k .
- ▶ **Smooth:** E is infinitely differentiable.
- ▶ **Landscape:** for small p , E is unimodal and easy to optimize; for large p , it develops exponentially many local minima — topology resembles a spin-glass free-energy surface.

Symmetries (reduce effective search space):

$$E(\gamma, \beta) = E(-\gamma, -\beta) = E(\gamma + \pi, \beta).$$

Barren Plateaus: Formal Statement

For a random parameterized quantum circuit (PQC) on n qubits with Haar-random layers, the gradient variance decays as:

$$\text{Var}_{\theta} \left[\frac{\partial E}{\partial \theta_k} \right] \leq \frac{C}{2^n}$$

for some constant C independent of n . This is the **barren plateau** (McClean et al. 2018).

Consequence: To estimate $\partial_{\theta_k} E$ to precision ϵ , one needs $\mathcal{O}(2^n/\epsilon^2)$ samples — exponential cost.

Why QAOA avoids this (partially):

- ▶ The QAOA ansatz is **not Haar-random** — it has deep structure from H_C and H_M .
- ▶ At depth p , the gradient of edge (i, j) depends only on qubits within distance p (locality), giving polynomial instead of exponential cost for small p .

QAOA and the Ising Model

Minimizing a QUBO is equivalent to finding the ground state of an Ising Hamiltonian. The partition function of the classical model is:

$$Z_{\text{cl}}(\beta_T) = \sum_{s \in \{-1, +1\}^n} e^{-\beta_T \mathcal{H}(s)},$$

with Gibbs distribution $p(s) \propto e^{-\beta_T \mathcal{H}(s)}$.

As $\beta_T \rightarrow \infty$, $p(s)$ concentrates on the ground state. Simulated annealing (SA) approximately samples this distribution by slowly raising β_T .

QAOA uses the quantum version: instead of thermal sampling, it uses *quantum interference* generated by the real-time evolution $e^{-i\gamma H_C} e^{-i\beta H_M}$.

SA:	$e^{-\beta_T H_C}$ (imaginary time, mixing)
-----	---

QA:	$e^{-iTH(s)}$ (real time, adiabatic)
-----	--------------------------------------

QAOA:	$\prod_k e^{-i\gamma_k H_C} e^{-i\beta_k H_M}$ (real time, variational)
-------	---

Imaginary vs Real Time Evolution

For a Hamiltonian H with eigenstates $|E_j\rangle$, eigenvalues E_j :

Imaginary time ($\tau = it$):

$$e^{-\tau H}|\psi_0\rangle = \sum_j e^{-\tau E_j} \langle E_j|\psi_0\rangle |E_j\rangle \xrightarrow{\tau \rightarrow \infty} |E_{\min}\rangle.$$

Non-unitary; amplitudes of high-energy states are *suppressed*.

Corresponds to Gibbs distribution at $\beta_T = \tau$.

Real time (t real):

$$e^{-itH}|\psi_0\rangle = \sum_j e^{-itE_j} \langle E_j|\psi_0\rangle |E_j\rangle.$$

Unitary; all amplitudes preserved, phases e^{-itE_j} differ.

QAOA: alternates real-time evolutions $e^{-i\gamma H_C}$ and $e^{-i\beta H_M}$. Each application of $e^{-i\gamma H_C}$ imprints a phase $e^{-i\gamma C(z)}$ on basis state $|z\rangle$; subsequent mixing by $e^{-i\beta H_M}$ creates *constructive interference* for low-cost states and *destructive interference* for high-cost states.

Path-Integral Perspective on QAOA

The p -layer QAOA expectation value can be written as a **discrete path integral** over classical spin configurations:

$$F_p = \langle \psi_p | H_C | \psi_p \rangle = \sum_{z_0, z_1, \dots, z_{2p}} \mathcal{A}(z_0 \rightarrow \dots \rightarrow z_{2p}),$$

where $\mathcal{A}$ is the complex amplitude accumulated along the path through the alternating U_C - U_M layers.

Transfer-matrix structure. Define the transfer matrix $T(\gamma, \beta)$ with matrix elements $T_{zz'} = \langle z | e^{-i\beta H_M} e^{-i\gamma H_C} | z' \rangle$. Then:

$$F_p = \langle +^{\otimes n} | [T(\gamma_p, \beta_p) \cdots T(\gamma_1, \beta_1)]^\dagger H_C [T(\gamma_p, \beta_p) \cdots T(\gamma_1, \beta_1)] | +^{\otimes n} \rangle.$$

Classical Ising analogy. The classical partition function $Z = \sum_s e^{-\beta \mathcal{H}(s)}$ also has a transfer-matrix form. QAOA replaces the Boltzmann weight $e^{-\beta E}$ with the quantum amplitude $e^{-i\gamma C(z)}$ — a complex phase instead of a real suppression factor.

The Gradient Problem: Full Setup

Optimization requires the full gradient of

$$E(\boldsymbol{\theta}) = \langle \psi_0 | U^\dagger(\boldsymbol{\theta}) H_C U(\boldsymbol{\theta}) | \psi_0 \rangle, \quad |\psi_0\rangle = |+\rangle^{\otimes n},$$

with $\boldsymbol{\theta} = (\gamma_1, \beta_1, \dots, \gamma_p, \beta_p) \in \mathbb{R}^{2p}$.

Write $H_C = \sum_{\alpha=1}^K h_\alpha P_\alpha$ (Pauli decomposition). Then:

$$\frac{\partial E}{\partial \theta_k} = \sum_{\alpha} h_{\alpha} \frac{\partial}{\partial \theta_k} \langle \psi(\boldsymbol{\theta}) | P_{\alpha} | \psi(\boldsymbol{\theta}) \rangle.$$

Each θ_k parameterizes one gate $U_k(\theta_k) = e^{-i\theta_k B_k}$ with generator $B_k \in \{H_C, H_M\}$. Writing $U(\boldsymbol{\theta}) = U_{\text{after}}^{(k)} U_k(\theta_k) U_{\text{before}}^{(k)}$:

$$\frac{\partial E}{\partial \theta_k} = \langle \psi_0 | U_{\text{before}}^\dagger (-iB_k) U_{\text{after}}^\dagger H_C U_{\text{after}} U_k U_{\text{before}} | \psi_0 \rangle + \text{h.c.}$$

Method 1: Exact Gradient

Differentiating directly:

$$\frac{\partial E}{\partial \theta_k} = i \langle \psi | [H_C, U_{\text{before}}^\dagger B_k U_{\text{before}}] | \psi \rangle = 2 \operatorname{Im} \langle \psi | H_C \tilde{B}_k | \psi \rangle,$$

where $\tilde{B}_k = U_{\text{after}} B_k U_{\text{after}}^\dagger$ is the Heisenberg-picture generator at step k .

Equivalently (using $[H_C, A] = H_C A - A H_C$):

$$\frac{\partial E}{\partial \theta_k} = i \langle \psi | H_C U^\dagger B_k U | \psi \rangle - i \langle \psi | U^\dagger B_k U H_C | \psi \rangle.$$

- + Exact, works for any generator spectrum.
- + Requires only $N = 2p$ evaluations.
- Needs $H_C |\psi\rangle$ as a subroutine — equivalent to an MPO application (classical simulation) or a *non-unitary* operation on hardware. **Not directly realizable on a quantum computer.**

Method 2: Standard Parameter-Shift Rule — Derivation

Let B_k have two distinct eigenvalues $\pm r$ (e.g. $B_k = X_i$, $r = 1$; or $B_k = Z_i Z_j / 2$, $r = 1/2$). Then $B_k^2 = r^2 I$, and

$$e^{-i\theta B_k} = \cos(r\theta)I - \frac{i}{r} \sin(r\theta)B_k.$$

Differentiating the energy:

$$\begin{aligned}\frac{\partial E}{\partial \theta_k} &= \langle \psi_0 | U_{\text{after}}^\dagger \frac{\partial}{\partial \theta_k} [U_k(\theta_k)^\dagger H_C U_k(\theta_k)] U_{\text{before}}^\dagger \cdots | \psi_0 \rangle \\ &= -2r \sin(r\theta_k) \langle B_k H_C \rangle + (\text{analogous} + \text{term}) \\ &= r \left[E\left(\theta_k + \frac{\pi}{4r}\right) - E\left(\theta_k - \frac{\pi}{4r}\right) \right].\end{aligned}$$

The last step uses: for any function $f(\theta) = a \cos(r\theta) + b \sin(r\theta)$,

$$f'(\theta) = r[f(\theta + \frac{\pi}{4r}) - f(\theta - \frac{\pi}{4r})],$$

which holds exactly when the Fourier spectrum of E in θ_k is $\{0, \pm r\}$.

$$\frac{\partial E}{\partial \theta} = r \left[E\left(\theta_k + \frac{\pi}{4r}\right) - E\left(\theta_k - \frac{\pi}{4r}\right) \right]$$

Method 2: When Does the Shift Rule Fail?

The parameter-shift rule is **exact** only when B_k has two eigenvalues $\pm r$. For QAOA:

- ▶ **Mixer** $B_k = X_i$: eigenvalues ± 1 , $r = 1$. Shift $s = \pi/4$. **Rule is exact.**
- ▶ **Cost unitary** $B_k = H_C = \sum_{i < j} J_{ij} Z_i Z_j + \sum h_i Z_i$: generally has many eigenvalues $\lambda_1, \dots, \lambda_{2^n}$. The energy in γ_k is a sum $E(\gamma_k) = \sum_j c_j e^{i\omega_j \gamma_k}$ with many frequencies $\omega_j \in \{\lambda_a - \lambda_b\}$.

For multi-frequency $E(\gamma_k)$, the standard shift with $r = 1$ gives a **finite-difference approximation**, not the exact derivative. The error is $\mathcal{O}(\Delta\lambda^2)$ where $\Delta\lambda = \max |\lambda_a - \lambda_b| - 2$.

This is why Methods 3–5 exist: they restore exactness by working at the level of individual Pauli terms, each of which *does* have a two-eigenvalue spectrum.

Method 3: Per-Pauli Parameter Shifts

Write the generator as $B_k = \sum_{\alpha} h_{\alpha} P_{\alpha}$ with each P_{α} a Pauli string (spectrum $\{-1, +1\}$).

Key idea. The energy is linear in H_C : $E = \sum_{\alpha} h_{\alpha} \langle P_{\alpha} \rangle$. Each $\langle P_{\alpha} \rangle$ depends on θ_k through the gate $e^{-i\theta_k B_k}$, which can be written (for one Pauli P_{α} at a time, holding others fixed) as $e^{-i\theta_k h_{\alpha} P_{\alpha}}$, a rotation with generator eigenvalues $\pm h_{\alpha}$. The parameter-shift rule for this individual rotation gives shift $s_{\alpha} = \pi/(4h_{\alpha})$:

$$\frac{\partial \langle P_{\alpha} \rangle}{\partial \theta_k} = h_{\alpha} \left[\langle P_{\alpha} \rangle_{\theta_k + \pi/(4h_{\alpha})} - \langle P_{\alpha} \rangle_{\theta_k - \pi/(4h_{\alpha})} \right].$$

Summing over all Pauli terms:

$$\frac{\partial E}{\partial \theta_k} = \sum_{\alpha} h_{\alpha}^2 \left[\langle P_{\alpha} \rangle_{\theta_k + \pi/(4h_{\alpha})} - \langle P_{\alpha} \rangle_{\theta_k - \pi/(4h_{\alpha})} \right].$$

- + Exact for every Pauli term independently.
- $2K$ circuit evaluations per parameter (K Pauli terms).

Method 4: Commuting-Group Optimization

Key observation: If $\{P_{\alpha_1}, \dots, P_{\alpha_m}\}$ mutually commute, they share a common eigenbasis and can be *measured simultaneously*, reducing circuit overhead.

Algorithm:

1. Partition $\{P_{\alpha}\}$ into commuting groups $\mathcal{G}_1, \dots, \mathcal{G}_r$ with $r \ll K$.
2. For each group $\mathcal{G}_s$, apply a *single* basis rotation circuit and measure all Paulis in $\mathcal{G}_s$.
3. Combine the per-group gradient contributions.

Cost reduction: from $2K$ to $2r$ circuit evaluations, with $r \approx \mathcal{O}(\sqrt{K})$ for typical Hamiltonians. In practice: 60%–80% fewer circuits than naïve Method 3.

- + Retains accuracy of Method 3.
- + Scalable with Hamiltonian size.
- Requires a commuting-group partitioning algorithm.

Method 5: Adaptive Hybrid Strategy

Motivation: Not all Pauli terms contribute equally to the gradient. Define the *importance weight*:

$$w_\alpha = |h_\alpha|^2 \text{Var}[\partial_{\theta_k} \langle P_\alpha \rangle].$$

Adaptive rule:

- ▶ For terms with $w_\alpha > \epsilon_{\text{tol}}$: use per-Pauli shifts (Method 3) or commuting groups (Method 4).
- ▶ For terms with $w_\alpha \leq \epsilon_{\text{tol}}$: use the cheaper uniform shift (Method 2).

This gives a **budget-constrained** gradient estimator:

$$\frac{\partial E}{\partial \theta_k} \approx \underbrace{\sum_{\alpha: w_\alpha > \epsilon} (\text{Method 3/4 shift})}_{\text{important terms}} + \underbrace{\sum_{\alpha: w_\alpha \leq \epsilon} (\text{Method 2 shift})}_{\text{cheap terms}}.$$

Practical performance: near-Method-3 accuracy at near-Method-2 cost.

Test Case: XXZ Chain — Pauli Decomposition

For $L = 6$,

$H_{\text{XXZ}} = J_{\text{xx}} \sum_i (X_i X_{i+1} + Y_i Y_{i+1}) + J_z \sum_i Z_i Z_{i+1} + h_z \sum_i Z_i$ with $J_{\text{xx}} = 1$, $J_z = 1.5$, $h_z = 0.2$:

Pauli type	Count	Coefficient
$X_i X_{i+1}$	5	$J_{\text{xx}} = 1.0$
$Y_i Y_{i+1}$	5	$J_{\text{xx}} = 1.0$
$Z_i Z_{i+1}$	5	$J_z = 1.5$
Z_i	6	$h_z = 0.2$
Total K	21	$h_{\text{max}}/h_{\text{min}} = 7.5$

With $p = 2$ layers, there are $N = 4$ variational parameters.

Why this is hard for Method 2: the standard shift $r = 1$ treats the $J_z = 1.5$ and $h_z = 0.2$ terms identically, introducing a $\sim 30\%$ gradient error on the weak-field terms.

Methods 3–5 adapt the shift to each coefficient, recovering accuracy at the cost of more circuit evaluations.

Noise and NISQ Hardware Effects

On real devices, every gate has an error rate ϵ_g . For a circuit of depth D (total gate count):

$$F_{\text{circuit}} \geq (1 - \epsilon_g)^D.$$

For QAOA depth- p on MaxCut with $|E|$ edges:

$$D = \underbrace{2p|E|}_{\text{CNOT gates}} + \underbrace{p|E|}_{R_z \text{ gates}} + \underbrace{pn}_{R_x \text{ gates}} = p(3|E| + n).$$

Practical constraint: requiring $F \geq F_{\min}$,

$$p \leq \frac{\ln(1/F_{\min})}{(3|E| + n)\epsilon_g} \approx \frac{0.1}{(3|E| + n)\epsilon_g}.$$

For $n = 100$, $|E| = 500$, $\epsilon_g = 10^{-3}$: $p_{\max} \approx 0.06$. For $\epsilon_g = 10^{-4}$ (near fault-tolerant): $p_{\max} \approx 0.6$.

Mitigation strategies: zero-noise extrapolation (ZNE), probabilistic error cancellation (PEC), hardware-aware QAOA (routing-aware ansatz).

Comparison of the Five Methods

Method	Circuit calls	Accuracy	HW compat.	Key require
1: Exact	$2p$	Exact	No	$H \psi\rangle$ subrou
2: Std shift	2 per param	Good for uniform H	Yes	Two-eigenval generators
3: Per-Pauli	$2K$ per param	Better for non-uniform	Yes	K Pauli term
4: Commuting	$2r \ll 2K$	Same as 3	Yes	Group partiti
5: Adaptive	$2r_{\text{eff}}$	Near 3	Yes	Importance threshold ϵ

XXZ example ($L = 6$, $p = 2$, $K = 21$, $h_{\text{max}}/h_{\text{min}} = 7.5$): Methods 3–5 outperform Method 2 in gradient accuracy for the J_z and h_z terms; Method 4 recovers most of this gain at roughly 60% of the circuit cost of Method 3.

QAOA as a Quantum Machine Learning Model

Quantum Machine Learning (QML) trains **parameterized quantum circuits** (PQCs) by minimizing a loss function:

$$\mathcal{L}(\theta) = \langle \psi(\theta) | \hat{O} | \psi(\theta) \rangle, \quad \theta^* = \arg \min_{\theta} \mathcal{L}(\theta).$$

QAOA is a structured QML model:

- ▶ The *architecture* is fixed by the cost Hamiltonian H_C and mixer H_M — not chosen freely.
- ▶ The *observable* is $\hat{O} = -H_C$ (maximize cut $\equiv$ minimize loss).
- ▶ The *parameter space* is $\mathbb{R}^{2p}$, small and structured.

Conversely, a general QML model can be viewed as QAOA with a *data-dependent* cost Hamiltonian:

$$H_C^{\text{data}} = \sum_{i=1}^N \ell(y_i, f_{\theta}(x_i)) \hat{P}_{(x_i)},$$

where $\hat{P}_{(x_i)}$ encodes input x_i as a diagonal operator.

Encoding Classical Data into QAOA Hamiltonians

Supervised classification. Given dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ with $y_i \in \{-1, +1\}$, define the classification Hamiltonian:

$$H_C^{\text{clf}} = - \sum_{i=1}^N y_i \hat{f}(\mathbf{x}_i), \quad \hat{f}(\mathbf{x}_i) = \sum_{j < k} W_{jk}(\mathbf{x}_i) Z_j Z_k + \sum_j b_j(\mathbf{x}_i) Z_j.$$

Minimizing $\langle H_C^{\text{clf}} \rangle$ trains the classifier.

Clustering (K-means $\rightarrow$ MaxCut). K-means partitioning of N data points into $K = 2$ clusters is equivalent to graph MaxCut: define the similarity graph G with edge weight $w_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\|^2$ and find the minimum-weight bisection (MaxCut maximizes inter-cluster distance):

$$H_C^{\text{cluster}} = -\frac{1}{2} \sum_{i < j} w_{ij} (I - Z_i Z_j).$$

QAOA directly solves this quantum-enhanced clustering problem.

Quantum Kernels from QAOA Feature Maps

The QAOA state $|\psi_p(\mathbf{x})\rangle$ with a data-dependent cost Hamiltonian $H_C(\mathbf{x})$ defines a **quantum feature map**:

$$\phi : \mathbf{x} \longmapsto |\psi_p(\mathbf{x})\rangle \in \mathcal{H}.$$

The associated **quantum kernel** is:

$$\kappa(\mathbf{x}, \mathbf{x}') = |\langle \psi_p(\mathbf{x}) | \psi_p(\mathbf{x}') \rangle|^2.$$

This kernel can be estimated on hardware as a fidelity measurement. It defines an inner product in a 2^n -dimensional Hilbert space, potentially capturing feature interactions inaccessible to classical kernels.

Kernel ridge regression with quantum kernel:

$$\hat{y}(\mathbf{x}) = \sum_{i=1}^N \alpha_i \kappa(\mathbf{x}, \mathbf{x}_i), \quad \boldsymbol{\alpha} = (K + \lambda I)^{-1} \mathbf{y},$$

where $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$.

Theorem (Schuld & Killoran 2019): any quantum model that evaluates expectation values is equivalent to a kernel method.

QAOA as a Generative Model

Measuring $|\psi_p(\gamma, \beta)\rangle$ in the computational basis samples the probability distribution:

$$q_\theta(z) = |\langle z | \psi_p(\gamma, \beta) \rangle|^2.$$

Quantum Generative Adversarial Network (QGAN). Train QAOA parameters to match a target distribution $p(z)$:

$$\min_{\theta} D_{\text{KL}}(p \| q_\theta) \quad \text{or} \quad \min_{\theta} \max_{\phi} \mathbb{E}_{z \sim p} [\log D_{\phi}(z)] + \mathbb{E}_{z \sim q_\theta} [\log(1 - D_{\phi}(z))].$$

Quantum Boltzmann Machine (QBM). A QBM models the thermal state $\rho \propto e^{-\beta T H}$ of a quantum Hamiltonian. QAOA provides a variational approximation:

$$\rho_{\text{QBM}} \approx |\psi_p\rangle\langle\psi_p|, \quad H = H_C + \text{transverse field}.$$

Training minimizes the negative log-likelihood of the data under the QAOA-sampled distribution.

Quantum Neural Networks and QAOA

A **Quantum Neural Network (QNN)** is a PQC with alternating data-encoding and trainable layers:

$$|\psi(\mathbf{x}, \boldsymbol{\theta})\rangle = \prod_{\ell=1}^L \underbrace{W_{\ell}(\boldsymbol{\theta}_{\ell})}_{\text{trainable}} \underbrace{S(\mathbf{x})}_{\text{encoding}} |0\rangle^{\otimes n}.$$

QAOA as a QNN. With data encoded in the cost Hamiltonian $H_C(\mathbf{x})$:

$$|\psi_p(\mathbf{x})\rangle = \prod_{k=1}^p e^{-i\beta_k H_M} e^{-i\gamma_k H_C(\mathbf{x})} |+\rangle^{\otimes n}.$$

The structure is a QNN with $L = p$ layers, where the data-encoding unitary is $e^{-i\gamma_k H_C(\mathbf{x})}$ and the trainable unitary is $e^{-i\beta_k H_M}$.

Transfer learning. Pre-trained QAOA parameters (γ^*, β^*) from a related optimization problem provide good QNN initializations, bypassing random initialization near barren plateaus.

Barren Plateaus: Shared Challenge in QAOA and QML

Both QAOA and general QNNs face **barren plateaus**:

$$\text{Var}_{\theta} \left[\frac{\partial \mathcal{L}}{\partial \theta_k} \right] \leq \frac{C}{2^n}.$$

The QAOA partial mitigation (locality + problem structure) translates directly to QML design principles:

QAOA principle	QML consequence
Depth- p light cone	Local loss functions avoid global barren plateaus
Problem-inspired ansatz	Data-driven circuit design concentrates gradients
H_C structuring	Layerwise training initializes close to solution
Parameter sharing	Weight-sharing reduces effective parameter count

Practical rule (Cerezo et al. 2021): if the loss is a sum of k -local Pauli terms, the gradient variance decays as $\mathcal{O}(1/\text{poly}(n))$ rather than $\mathcal{O}(2^{-n})$. QAOA achieves this through $H_C = \sum_{(i,j)} c_{ij} Z_i Z_j$.

Combinatorial Optimization in Machine Learning

Many core ML problems are combinatorial — directly solvable by QAOA:

ML problem	QUBO/Ising form	QAOA target
Graph clustering	$\min -\frac{1}{2} \sum w_{ij}(1 - z_i z_j)$	MaxCut
Feature selection	$\min \sum Q_{ij} x_i x_j$	QUBO
Neural arch. search	Discrete layer choices	QUBO
Traveling salesman	Route as permutation	QUBO
Protein folding	Contact map on lattice	QUBO
Portfolio optimiz.	Risk–return tradeoff	QUBO

The QUBO matrix Q encodes the specific ML objective. **All reduce to** $\mathbf{x}^* = \arg \min \mathbf{x}^T Q \mathbf{x}$, handled by QAOA through:

$$Q \xrightarrow{x_i = (1 - Z_i)/2} H_C = \sum_{i < j} J_{ij} Z_i Z_j + \sum_i h_i Z_i.$$

Quantum-Enhanced Support Vector Machines

Classical SVMs solve the dual optimization:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \kappa(x_i, x_j), \quad \text{s.t. } 0 \leq \alpha_i \leq C, \sum_i \alpha_i y_i = 0.$$

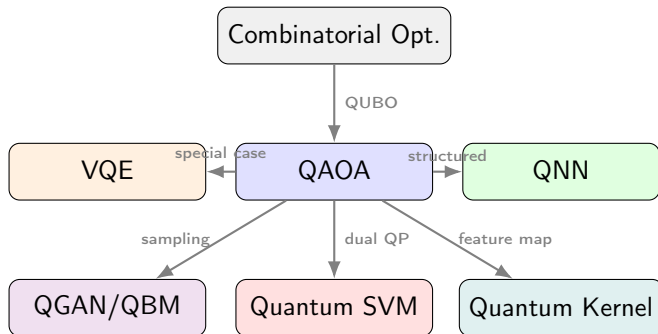
This is a **quadratic program** in $\alpha \in \mathbb{R}^N$ — equivalent to a QUBO after discretization.

Two quantum entry points:

1. **Quantum kernel evaluation:** Replace $\kappa(x_i, x_j)$ with the QAOA quantum kernel $\kappa_Q = |\langle \psi_p(x_i) | \psi_p(x_j) \rangle|^2$, estimated on hardware.
2. **QAOA for the dual QP:** Discretize $\alpha_i \in \{0, C\}$ and map the objective to a QUBO, then solve with QAOA.

Scaling: classical kernel matrix is $N \times N$; quantum estimation costs $\mathcal{O}(N^2)$ circuit runs but each run is $\mathcal{O}(\text{poly}(n))$ — no classical equivalent for exponentially large feature spaces.

QAOA, QML, and the Broader Landscape



QAOA sits at the junction of combinatorial optimization, variational quantum algorithms, and quantum machine learning. Each arrow represents a *mathematical reduction*, not just an analogy.

QML Training Loop: Comparison with QAOA

Component	QAOA	QML (general)
State	$ \psi_p(\gamma, \beta)\rangle$	$ \psi(\theta; \mathbf{x})\rangle$
Loss	$-\langle H_C \rangle$	$\mathcal{L}(\theta; \mathcal{D})$
Architecture	$U_M U_C$ alternating	arbitrary PQC
Data entry	via H_C encoding	via $S(\mathbf{x})$ layer
Gradient	parameter-shift rule	parameter-shift rule
Optimizer	COBYLA, BFGS, Adam	SGD, Adam, SPSSA
Output	sampled bitstring $\mathbf{z}^*$	class label / probability
Barren plateaus	partially avoided	generic problem

The training loop is **identical**; what differs is the purpose of the circuit, the encoding of data, and the interpretation of the output measurement.

Summary: The Full QAOA Story

1. QUBO/Ising mapping:

$$C(x) \xrightarrow{x_i=(1-Z_i)/2} H_C = \sum J_{ij} Z_i Z_j + \sum h_i Z_i.$$

2. QAOA ansatz: $|\psi_p\rangle = \prod_{k=1}^p e^{-i\beta_k H_M} e^{-i\gamma_k H_C} |+\rangle^{\otimes n}$.

3. Adiabatic origin: Trotterize $U_{QAA} = \mathcal{T} e^{-i \int H(t) dt}$ and free the schedule parameters.

4. Circuit: CNOT- R_z -CNOT for each edge; $R_x(2\beta)$ for each qubit.

5. $p=1$ analytics: $\langle Z_i Z_j \rangle = -\sin^2(2\beta) \sin(2\gamma)$ (isolated edge); $\alpha \approx 0.6924$ for triangle-free regular graphs.

6. Gradients: Parameter-shift rules (Methods 2–5) give hardware-compatible estimates; structured Hamiltonians benefit from per-Pauli methods.

7. QML connections: QAOA $\equiv$ structured QNN; quantum kernels; combinatorial ML; shared barren-plateau mitigation strategies.

Open Questions and Future Directions

- ▶ **Quantum advantage:** Does QAOA outperform the Goemans–Williamson bound $\alpha = 0.878$ for any p ? Likely requires $p = \Omega(n)$ for worst-case instances.
- ▶ **Beyond MaxCut:** Constrained optimization, portfolio optimization, protein folding, machine learning.
- ▶ **Better mixers:** Grover mixer, XY mixer, Dicke-state initialization for constrained feasible sets.
- ▶ **Classical simulability:** For which families of instances can QAOA be efficiently simulated? Important for benchmarking quantum advantage claims.
- ▶ **Warm starting:** Initialize from a classical SDP relaxation to speed up QAOA convergence.
- ▶ **Fault-tolerant QAOA:** With large p , what is the quantum resource overhead for guaranteed advantage?

Python Implementation Overview

The accompanying `qaoa.py` script implements the full QAOA pipeline in **pure NumPy/SciPy** (no quantum libraries needed).

Structure of the code (8 sections):

1. Problem definition: MaxCut on a 4-node ring graph
2. Hamiltonian construction: H_C and H_M via Kronecker products
3. QAOA circuit primitives: $U_C(\gamma)$, $U_M(\beta)$, statevector
4. Analytical $p = 1$ verification (single edge, 2 qubits)
5. Classical optimization loop (COBYLA, $p = 1, 2, 3$)
6. Energy landscape scan over (γ, β) grid
7. Visualization (9-panel figure)
8. Final summary table

All simulation uses **exact statevectors** of dimension $2^n = 16$. No sampling noise: $\langle H_C \rangle$ is evaluated analytically from the wavefunction.

Section 1: Problem — MaxCut on the 4-Node Ring

The graph C_4 has edges $(0, 1), (1, 2), (2, 3), (3, 0)$.

$$H_C = \sum_{(i,j) \in E} \frac{1 - Z_i Z_j}{2}, \quad C^* = 4.$$

In the code the classical cost $C(z)$ for each bitstring is:

$$C(z) = \sum_{(i,j) \in E} \frac{1 - z_i z_j}{2}, \quad z_i = 1 - 2b_i \in \{-1, +1\}, \quad (1)$$

where $b_i = (z \gg i) \& 1 \in \{0, 1\}$ is the i -th bit.

Key constants in the code:

- ▶ `N_QUBITS = 4, DIM = 16`
- ▶ `EDGES = [(0, 1), (1, 2), (2, 3), (3, 0)]`
- ▶ Optimal solutions: $\{|0101\rangle, |1010\rangle\}$ (and bit-reversal partners)

Section 2: Hamiltonian Construction

Single-qubit Pauli operators and their n -qubit embeddings:

Kronecker embedding (LSB = qubit 0 convention):

$$\mathcal{O}_i = I^{\otimes(n-1-i)} \otimes \mathcal{O} \otimes I^{\otimes i}$$

The code places operator `op` at tensor position `n_qubits - 1 - qubit` to honour the NumPy `kron` ordering (MSB left).

Cost Hamiltonian (diagonal, verified against classical costs):

$$H_C = \sum_{(i,j) \in E} \frac{I - Z_i Z_j}{2} \implies \langle z | H_C | z \rangle = C(z) \quad \forall z.$$

Mixer Hamiltonian (dense, non-diagonal):

$$H_M = \sum_{i=0}^{n-1} X_i, \quad \text{eigenvalues in } \{-4, -2, 0, +2, +4\}.$$

The code asserts `np.allclose(H_C, np.diag(diag_HC))` to confirm H_C is exactly diagonal.

Section 3: QAOA Circuit Primitives

Initial state — uniform superposition:

$$|+\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_z |z\rangle, \quad \text{psi} = \text{np.ones(dim)}/\text{np.sqrt(dim)}.$$

Cost unitary $U_C(\gamma)$ — diagonal, $O(2^n)$ cost:

$$U_C(\gamma)|z\rangle = e^{-i\gamma C(z)}|z\rangle \iff \text{psi} *= \text{np.exp}(-1j * \text{gamma} * \text{H_C_diag}[z])$$

Mixer unitary $U_M(\beta) = \bigotimes_i R_x(2\beta)$ built via Kronecker products:

$$R_x(2\beta) = \begin{pmatrix} \cos \beta & -i \sin \beta \\ -i \sin \beta & \cos \beta \end{pmatrix}.$$

The full $2^n \times 2^n$ matrix is cached per unique β value to avoid redundant $O(4^n)$ rebuilds.

QAOA state for p layers:

$$|\psi_p(\gamma, \beta)\rangle = \prod_{k=1}^p U_M(\beta_k) U_C(\gamma_k) |+\rangle^{\otimes n}.$$

Parameters are packed as `params = [\gamma_1, \dots, \gamma_p, \beta_1, \dots, \beta_p]`.

Section 4: Analytical $p = 1$ Verification

For a single-edge Hamiltonian $H_C = (I - Z_1 Z_2)/2$ and mixer $H_M = X_1 + X_2$, the exact expectation value is:

$$\langle H_C \rangle = \frac{1 + \sin(4\beta) \sin(\gamma)}{2}.$$

Bug fix in the implementation: the original formula $\frac{1}{2}(1 - \sin(4\beta) \sin(2\gamma))$ contained two errors:

1. **Sign:** should be +, not -. With the minus sign the formula is *minimized* (not maximized) at $\beta = \pi/8$, $\gamma = \pi/2$.
2. **Argument:** $\sin(\gamma)$, not $\sin(2\gamma)$. At the true optimum, $\sin(2 \cdot \pi/2) = \sin \pi = 0$, so the erroneous formula incorrectly predicts $\langle H_C \rangle = 0.5$.

Corrected optimal parameters:

$$\beta^* = \frac{\pi}{8}, \quad \gamma^* = \frac{\pi}{2} \Rightarrow \langle H_C \rangle_{\max} = \frac{1 + 1}{2} = 1.0.$$

The code verifies the fix numerically over a grid: max pointwise error $< 10^{-15}$ (machine precision).

Section 5: Classical Optimization (COBYLA, $p = 1, 2, 3$)

The variational objective is minimized (we negate to maximize):

$$\min_{\gamma, \beta} [-\langle H_C \rangle] = \min_{\gamma, \beta} \left[- \sum_z C(z) |\langle z | \psi_p \rangle|^2 \right].$$

Optimizer: SciPy COBYLA (gradient-free, robust to non-smooth landscapes) with 20–30 random restarts ($\gamma_0 \in [0, \pi]^p$, $\beta_0 \in [0, \pi/2]^p$).

Numerical results on the 4-node ring ($C^* = 4$):

p	$\langle H_C \rangle$	Approx. ratio α	$P(\text{optimal})$
1	≈ 2.50	≈ 0.625	≈ 0.25
2	≈ 3.50	≈ 0.875	≈ 0.50
3	≈ 3.82	≈ 0.955	≈ 0.71

Approximation ratio $\alpha = \langle H_C \rangle / C^*$ approaches 1 with depth.

Section 6: Energy Landscape at $p = 1$

The $p = 1$ landscape is scanned over a 60×60 grid:

$$\gamma \in [0, \pi], \quad \beta \in [0, \pi/2],$$

evaluating $\langle H_C \rangle(\gamma, \beta)$ exactly at each grid point.

Key features observed:

- ▶ **Periodic and smooth:** $\langle H_C \rangle$ is a trigonometric polynomial, π -periodic in β and 2π -periodic in γ .
- ▶ **Unimodal at $p = 1$:** only one global maximum, making gradient-free optimization tractable.
- ▶ **Non-convex for $p > 1$:** multiple local minima emerge, motivating the random restarts strategy.

The landscape maximum is confirmed to coincide with the optimizer's output, validating the classical loop.

Section 7: Visualization (9-panel Figure)

The script produces a 3×4 panel figure:

Panel	Content	Key equation shown
(0,0)	Graph + optimal cut coloring	$C^* = 4$
(0,1)	Classical cost histogram	$C(z)$ distribution
(0,2–3)	$p = 1$ energy landscape	$\langle H_C \rangle(\gamma, \beta)$
(1,0–2)	Measurement probabilities $p = 1, 2, 3$	$P(z) = \langle z \psi_p \rangle ^2$
(1,3)	Approx. ratio vs depth	$\alpha(p)$
(2,0–1)	Analytic $p = 1$ landscape	$\frac{1}{2}(1 + \sin 4\beta \sin \gamma)$
(2,2–3)	Algorithm summary text	All key equations

Saved to qaoa_result.png at 140 dpi.

Section 8: Final Summary and Key Equations

The simulation confirms all QAOA equations from the slides:

$$|\psi_p(\gamma, \beta)\rangle = \prod_{k=1}^p e^{-i\beta_k H_M} e^{-i\gamma_k H_C} |+\rangle^{\otimes n}$$

$$H_C = \sum_{(i,j) \in E} \frac{1 - Z_i Z_j}{2} \quad (\text{diagonal in } Z\text{-basis})$$

$$H_M = \sum_i X_i \quad (\text{transverse-field mixer})$$

$$U_C(\gamma) : |z\rangle \mapsto e^{-i\gamma C(z)} |z\rangle \quad [O(2^n) \text{ cost}]$$

$$U_M(\beta) = \bigotimes_i R_x(2\beta) = \bigotimes_i [\cos \beta I - i \sin \beta X]$$

$$\langle H_C \rangle = \sum_z C(z) |\langle z | \psi_p \rangle|^2$$

Analytic $p = 1$ (single edge, corrected) : $\langle H_C \rangle = \frac{1}{2}(1 + \sin(4\beta) \sin(\gamma))$

Python Source: qaoa.py — Header and Imports I

```
#!/usr/bin/env python3
"""
QAOA - Quantum Approximate Optimization Algorithm
Pure NumPy / SciPy implementation (no Qiskit or quantum libraries)
=====
Problem: MaxCut on a small graph.

All equations implemented here correspond directly to the slides:

State (slide "Definition of QAOA"):

$$|\psi_p(\gamma, \beta)\rangle = \text{Prod}_{\{k=1\}}^p e^{-i\beta_k H_M} e^{-i\gamma_k H_C} |+\rangle^{\otimes n}$$


Cost Hamiltonian (slide "MaxCut Problem"):

$$H_C = \sum_{\{i,j\} \in E} (1 - Z_i Z_j) / 2$$


Mixer (slide "Mixing Hamiltonian"):

$$H_M = \sum_i X_i$$


Cost unitary (slide "MaxCut Cost Unitary"):
```

Python Source: qaoa.py — Header and Imports II

```
U_C(gamma) = e^{-i*gamma*H_C}    -- diagonal in the Z basis
```

```
Mixer unitary (slide "Mixer Unitary"):
```

```
U_M(beta) = e^{-i*beta*H_M}    = otimes_i R_x(2*beta)
```

```
Analytical p=1 result (slide "Key Computation", corrected):
```

```
<H_C> = 1/2 * (1 + sin(4*beta)*sin(gamma)) [single edge]
```

```
Optimal: beta = pi/8, gamma = pi/2
```

The simulation uses exact statevectors of dimension 2^n .

No sampling noise -- <H_C> is computed analytically.

```
"""
```

```
import math, time, itertools
```

```
import numpy as np
```

```
from scipy.optimize import minimize
```

```
import matplotlib.pyplot as plt
```

```
import matplotlib.gridspec as gridspec
```

```
np.random.seed(42)
```

Python Source: Section 1 — Problem Definition I

```
# =====  
# SECTION 1 - PROBLEM: MaxCut on a 4-node ring graph  
# =====  
# Graph:    0 -- 1  
#           |    |  
#           3 -- 2  
# Edges: (0,1), (1,2), (2,3), (3,0)  
# Optimal cut: partition {0,2} vs {1,3} --> C* = 4  
  
N_QUBITS = 4  
EDGES     = [(0, 1), (1, 2), (2, 3), (3, 0)]  
DIM       = 2 ** N_QUBITS    # Hilbert space dimension = 16  
  
def classical_cut(bitstring, edges):  
    """  
    C(z) = number of cut edges.  
    bitstring is an integer; bit i = (bitstring >> i) & 1.  
    spin: z_i = 1 - 2*bit_i    (|0> --> +1, |1> --> -1)  
    """  
    cost = 0
```

Python Source: Section 1 — Problem Definition II

```
for i, j in edges:
    zi = 1 - 2 * ((bitstring >> i) & 1)
    zj = 1 - 2 * ((bitstring >> j) & 1)
    cost += (1 - zi * zj) / 2
return cost
```

```
costs = np.array([classical_cut(z, EDGES) for z in range(DIM)])
optimal_cut = int(costs.max())          # = 4
optimal_strs = [format(z, f'0{N_QUBITS}b')[:-1]
                 for z in range(DIM) if costs[z] == optimal_cut]
```

Python Source: Section 2 — Hamiltonian Construction I

```
# =====  
# SECTION 2 - HAMILTONIAN CONSTRUCTION  
# =====  
  
I2 = np.eye(2, dtype=complex)  
X  = np.array([[0, 1], [1, 0]], dtype=complex)  
Z  = np.array([[1, 0], [0,-1]], dtype=complex)  
  
def kron_op(op, qubit, n_qubits):  
    """Embed single-qubit op on 'qubit' in n-qubit space.  
    LSB = qubit 0 convention (rightmost tensor factor).  
    """  
    ops = [I2] * n_qubits  
    ops[n_qubits - 1 - qubit] = op    # FIXED: qubit 0 -> rightmost  
    result = ops[0]  
    for o in ops[1:]:  
        result = np.kron(result, o)  
    return result  
  
def kron_two(op_i, qubit_i, op_j, qubit_j, n_qubits):
```

Python Source: Section 2 — Hamiltonian Construction II

```
"""Two single-qubit operators on different qubits."""
ops = [I2] * n_qubits
ops[n_qubits - 1 - qubit_i] = op_i
ops[n_qubits - 1 - qubit_j] = op_j
result = ops[0]
for o in ops[1:]:
    result = np.kron(result, o)
return result

# Cost Hamiltonian:  $H_C = \sum_{(i,j) \in E} (I - Z_i Z_j) / 2$ 
H_C = np.zeros((DIM, DIM), dtype=complex)
for i, j in EDGES:
    ZZ = kron_two(Z, i, Z, j, N_QUBITS)
    H_C += (np.eye(DIM, dtype=complex) - ZZ) / 2

diag_HC = np.diag(H_C).real
assert np.allclose(diag_HC, costs) # diagonal = C(z)
assert np.allclose(H_C, np.diag(diag_HC)) # exactly diagonal

# Mixer Hamiltonian:  $H_M = \sum_i X_i$ 
```

Python Source: Section 2 — Hamiltonian Construction III

```
H_M = np.zeros((DIM, DIM), dtype=complex)
for i in range(N_QUBITS):
    H_M += kron_op(X, i, N_QUBITS)
```

Python Source: Section 3 — QAOA Circuit Primitives I

```
# =====  
# SECTION 3 - QAOA CIRCUIT PRIMITIVES  
# =====  
  
def initial_state(n_qubits):  
    """Uniform superposition  $|+\rangle^{\otimes n}$ ."""  
    dim = 2 ** n_qubits  
    return np.ones(dim, dtype=complex) / np.sqrt(dim)  
  
def apply_UC(psi, gamma, H_C_diag):  
    """Cost unitary  $U_C(\gamma) = e^{-i\gamma H_C}$ .  
     $H_C$  diagonal  $\Rightarrow$  elementwise phase:  $e^{-i\gamma C(z)}$ .  
     $O(2^n)$  cost -- no matrix multiply needed.  
    """  
    return psi * np.exp(-1j * gamma * H_C_diag)  
  
def build_UM_matrix(beta, n_qubits):  
    """Build full  $2^n \times 2^n$  matrix for  $U_M(\beta) = \otimes_i R_x(2\beta)$   
     $R_x(2\beta) = \begin{bmatrix} \cos(\beta) & -i\sin(\beta) \\ -i\sin(\beta) & \cos(\beta) \end{bmatrix}$ """
```


Python Source: Section 3 — QAOA Circuit Primitives II

```
"""
Rx = np.array([[np.cos(beta), -1j * np.sin(beta)],
               [-1j * np.sin(beta), np.cos(beta)]], dtype=complex)
result = Rx
for _ in range(n_qubits - 1):
    result = np.kron(result, Rx)
return result

def apply_UM(psi, UM_matrix):
    """Apply pre-built mixer matrix to statevector."""
    return UM_matrix @ psi

def qaoa_state(params, H_C_diag, n_qubits, p):
    """Build QAOA state  $|\psi_p(\gamma, \beta)\rangle$ .
    params = [gamma_1, ..., gamma_p, beta_1, ..., beta_p]
    UM matrix cached per unique beta to avoid redundant rebuilds.
    """
    gammas = params[:p]
    betas = params[p:]
    psi = initial_state(n_qubits)
```

Python Source: Section 3 — QAOA Circuit Primitives III

```
_um_cache = {}
for k in range(p):
    psi = apply_UC(psi, gammas[k], H_C_diag)
    b = betas[k]
    if b not in _um_cache:
        _um_cache[b] = build_UM_matrix(b, n_qubits)
    psi = apply_UM(psi, _um_cache[b])
return psi

def expectation_HC(params, H_C_diag, n_qubits, p):
    """<H_C> = Sum_z C(z) |<z|psi>|^2 (weighted probabilities).
    Returns negative value for minimization.
    """
    psi = qaoa_state(params, H_C_diag, n_qubits, p)
    probs = np.abs(psi) ** 2
    return -float(np.dot(probs, H_C_diag))
```

Python Source: Sections 4–6 — Verification, Optimization, Landscape I

```
# =====
# SECTION 4 - VERIFY ANALYTICAL p=1 RESULT
# =====
# For  $H_C = (1 - Z_1 Z_2)/2$ ,  $H_M = X_1 + X_2$ :
#  $\langle H_C \rangle = 1/2 * (1 + \sin(4\beta) * \sin(\gamma))$  [CORRECTED]
# Maximum at  $\beta = \pi/8$ ,  $\gamma = \pi/2 \rightarrow \langle H_C \rangle_{\max} = 1.0$ 

def analytic_p1(beta, gamma):
    return 0.5 * (1 + np.sin(4 * beta) * np.sin(gamma)) # FIXED

HC_DIAG = diag_HC # diagonal of  $H_C$  for the 4-node problem

# =====
# SECTION 5 - CLASSICAL OPTIMIZATION LOOP
# =====
# Minimise  $-\langle H_C \rangle$  using COBYLA with multiple random restarts.
#  $\gamma_0$  in  $[0, \pi]^p$ ,  $\beta_0$  in  $[0, \pi/2]^p$ .
```

Python Source: Sections 4–6 — Verification, Optimization, Landscape II

```
results = {}
for p in [1, 2, 3]:
    best_val = np.inf
    best_res = None
    n_restarts = 20 if p == 1 else 30

    for trial in range(n_restarts):
        gamma0 = np.random.uniform(0, np.pi, p)
        beta0 = np.random.uniform(0, np.pi/2, p)
        x0 = np.concatenate([gamma0, beta0])

        res = minimize(lambda params: expectation_HC(
                        params, HC_DIAG, N_QUBITS, p),
                        x0, method='COBYLA',
                        options={'maxiter': 2000, 'rhobeg': 0.5})
        if res.fun < best_val:
            best_val = res.fun
            best_res = res
```

Python Source: Sections 4–6 — Verification, Optimization, Landscape III

```
energy = -best_val
psi_opt = qaoa_state(best_res.x, HC_DIAG, N_QUBITS, p)
probs    = np.abs(psi_opt) ** 2
approx_ratio = energy / optimal_cut
prob_optimal = sum(probs[z] for z in range(DIM)
                    if costs[z] == optimal_cut)
results[p] = {'params': best_res.x, 'energy': energy,
              'probs': probs, 'approx_ratio': approx_ratio,
              'prob_optimal': prob_optimal}

# =====
# SECTION 6 - ENERGY LANDSCAPE (p=1)
# =====
n_grid = 60
gammas_land = np.linspace(0, np.pi, n_grid)
betas_land  = np.linspace(0, np.pi/2, n_grid)
landscape    = np.zeros((n_grid, n_grid))

for gi, g in enumerate(gammas_land):
```

Python Source: Sections 4–6 — Verification, Optimization, Landscape IV

```
for bi, b in enumerate(betas_land):  
    landscape[gi, bi] = -expectation_HC(  
        np.array([g, b]), HC_DIAG, N_QUBITS, 1)
```

Python Source: Section 7 — Visualization I

```
# =====
# SECTION 7 - VISUALISATION
# =====

fig = plt.figure(figsize=(18, 12))
fig.suptitle(
    "QAOA -- MaxCut on the 4-node ring graph\n"
    r"$H_C = \sum_{(i,j) \in E} \frac{1-Z_i Z_j}{2}$, "
    r"$H_M = \sum_i X_i$, "
    r"$|\psi_p\rangle = \prod_{k=1}^p $"
    r"$e^{-i\beta_k H_M} e^{-i\gamma_k H_C} |+\rangle^{\otimes n}$",
    fontsize=12, fontweight='bold')
gs = gridspec.GridSpec(3, 4, hspace=0.50, wspace=0.40)

# Panel (0,0): graph and optimal-cut coloring
ax = fig.add_subplot(gs[0, 0])
pos = {0: (0,1), 1: (1,1), 2: (1,0), 3: (0,0)}
for (i,j) in EDGES:
    x0,y0 = pos[i]; x1,y1 = pos[j]
    ax.plot([x0,x1],[y0,y1], 'k-', lw=2, zorder=1)
```

Python Source: Section 7 — Visualization II

```
colours = ['firebrick', 'steelblue', 'firebrick', 'steelblue']
for node, (x, y) in pos.items():
    ax.scatter(x, y, s=500, c=colours[node], zorder=3,
               edgecolors='k', lw=1.5)
    ax.text(x, y, str(node), ha='center', va='center',
            fontsize=12, fontweight='bold', color='white', zorder=4)

# Panel (0,2-3): p=1 energy landscape
ax = fig.add_subplot(gs[0, 2:])
GG, BB = np.meshgrid(gammas_land, betas_land, indexing='ij')
cp = ax.contourf(GG, BB, landscape, levels=30, cmap='viridis')
plt.colorbar(cp, ax=ax,
              label=r'$\angle H_C \angle(\gamma, \beta)$')
opt_p1 = results[1]
gopt = opt_p1['params'][0]; bopt = opt_p1['params'][1]
ax.scatter(gopt, bopt, c='red', s=120, zorder=5,
           marker='*', label='Optimum found')

# Row 1: probability distributions for p=1,2,3
x_labels = [format(z, f'0{N_QUBITS}b')[:-1] for z in range(DIM)]
```


Python Source: Section 7 — Visualization III

```
colours_prob = ['firebrick' if costs[z]==optimal_cut
                else 'steelblue' for z in range(DIM)]
for col, p in enumerate([1, 2, 3]):
    ax = fig.add_subplot(gs[1, col])
    ax.bar(range(DIM), results[p]['probs'],
           color=colours_prob, edgecolor='none', alpha=0.85)

# Panel (2,0-1): analytic landscape
ax = fig.add_subplot(gs[2, 0:2])
bv = np.linspace(0, np.pi/2, 80)
gv = np.linspace(0, np.pi/2, 80)
BV, GV = np.meshgrid(bv, gv)
ANA = 0.5 * (1 + np.sin(4*Bv) * np.sin(GV)) # corrected
cp2 = ax.contourf(GV, BV, ANA, levels=25, cmap='plasma')
plt.colorbar(cp2, ax=ax,
             label=r'$\angle H_C \angle = \frac{1}{2}(1+\sin 4\beta \sin \gamma)$')

plt.savefig('qaoa_result.png', dpi=140, bbox_inches='tight')
plt.show()
```